Individual Homework – 3

VAISHAK BALACHANDRA

Fall 2025
CS 51510-001 Algorithms
Purdue University Northwest
09/26/2025

OnlineGDB: *https://onlinegdb.com/AJT7alkS6 (Only Java)*

GitHub: *https://github.com/vaishakbalachandra/algorithms-hw3__Sorting_Algo_Memory_DB*

Name	Email
Vaishak Balachandra	**balachv@pnw.edu**

Table of Contents

Table of Figures

ABSTRACT

This project builds and runs a linked list-based toy memory database with sorting functions using two different programming languages, namely Java and Julia. The system uses recursion to import information from a CSV file into memory. It then sorts the data using two sorting algorithms, bubble sort and insertion sort, on the in-memory linked list before exporting the sorted dataset recursively into new CSV files. This work illustrates basic ideas of grouping, repetition, and data structure manipulations while contrasting two programming paradigms: the Julia's scientific computing style and Java's object-oriented approach. With appropriately sorted outputs and insertion sort exceeding bubble sorting with regards to of CPU use, the findings demonstrate that all of the task's requirements were successfully met. In contrast, disk utilization remained rather consistent among approaches.

I. INTRODUCTION

In Individual Homework-3, individuals add sorting capabilities in two various languages of programming to the basic "memory database" that was created in the previous assignment. By working on this project, which focuses on the integration of sorting, recursion, and linked list algorithms, students can improve their understanding of algorithm design and performance evaluation. It was necessary to complete three primary tasks to examine CPU and disk efficiency in an Ubuntu environment: first utilizing recursion to load a CSV file into memory as a linked list; second, using two conventional sorting algorithms (bubble sort and insertion sort) on the in-memory dataset; and third, using recursion to export the sort results back to CSV.

The Julia implementation was the task's primary focus. Julia and other high-level programming languages are widely used in numerical and scientific computation. Its clear syntax

makes CSV parsing simple and recursion easier. In this project, a linked list was recursively loaded with pupil information from a CSV file. In order to create bubble sort, node contents were swapped, whereas insertion sort necessitated relinking vertices into a sorted sublist. Finally, recursive export techniques were used to write the sorted outputs into new files (julia-bubble-sorted.csv and julia-insert-sorted.csv). The creation of a Java memory-based data with sorting was the second essential stage. Java was an object-oriented, strongly typed language with strong class structures for memory management, recursive utilities, and node construction.

Ultimately, an Ubuntu system was used to execute and evaluate both options. In addition to verifying cross-platform compatibility, this step made it possible to monitor CPU and disk use for each algorithm using Linux efficiency tools like /usr/bin/time -v, pidstat, or iostat. Screenshots of the execution showed that insertion sort performed better than sorting bubbles in terms of processing usage and that all project requirements were met.

In conclusion, by building on Homework-2 by adding sorting features to the toys memory database, Individual Homework-3 reinforced the concepts of linked lists, recurrent patterns, and algorithm analysis. The next sections describe the Julia & Java runs, show the Ubuntu run results, analyze the computation time and disk usage comparisons, and finally discuss the challenges faced and the lessons learned.

II. DEVELOP SORTING MEMORY DATABASE IN JULIA

The first implementation of the sorting memory database was developed using the Julia programming language. Julia was selected for its concise syntax, strong support for numerical computing, and straightforward approach to recursion. The Julia solution fulfills all three requirements of Homework-3:

1. Load the student CSV file into memory using recursion.
2. Perform bubble sort and insertion sort on the in-memory linked list.
3. Export the sorted data to a new CSV file through recursion.

The design was organized into a single driver file named main.jl, which contained the definitions for the node structure, loading functions, sorting algorithms, and recursive export. Unlike Homework-2, which emphasized add and delete operations, the Julia implementation in Homework-3 was centered on the two sorting algorithms and the measurement of their performance.

1. Load CSV File Using Recursion

The function build_list was created to process the input dataset (student-data.csv) and load its rows into the linked list recursively. Each row was appended to the list through successive recursive calls, building the entire structure node by node. At program startup, the system confirmed that the dataset had been loaded and displayed the initial record count. And here in the code, both loading and calling the algorithm, and as well as saving the file, will be performed using a main single command line as in Figure 1.

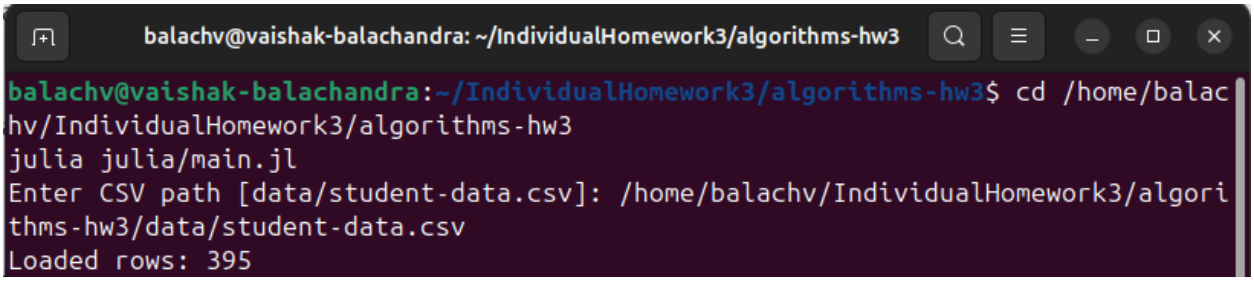


Figure 1- Julia terminal showing recursive loading of student-data.csv and row count confirmation

2. Bubble Sort Implementation

The Julia implementation of bubble sort repeatedly traversed the linked list, comparing adjacent nodes and swapping their row values when they were out of order. Because linked lists

do not support random access, node relinking was avoided, and swaps were handled by exchanging node data directly. Although bubble sort has a time complexity of $O(n^2)$, it provided a clear example of repeated recursive traversal in a pointer-based structure. The sorted results were written to julia-bubble-sorted.csv. Figure 2 shows the Julia terminal output after bubble sorting the dataset.

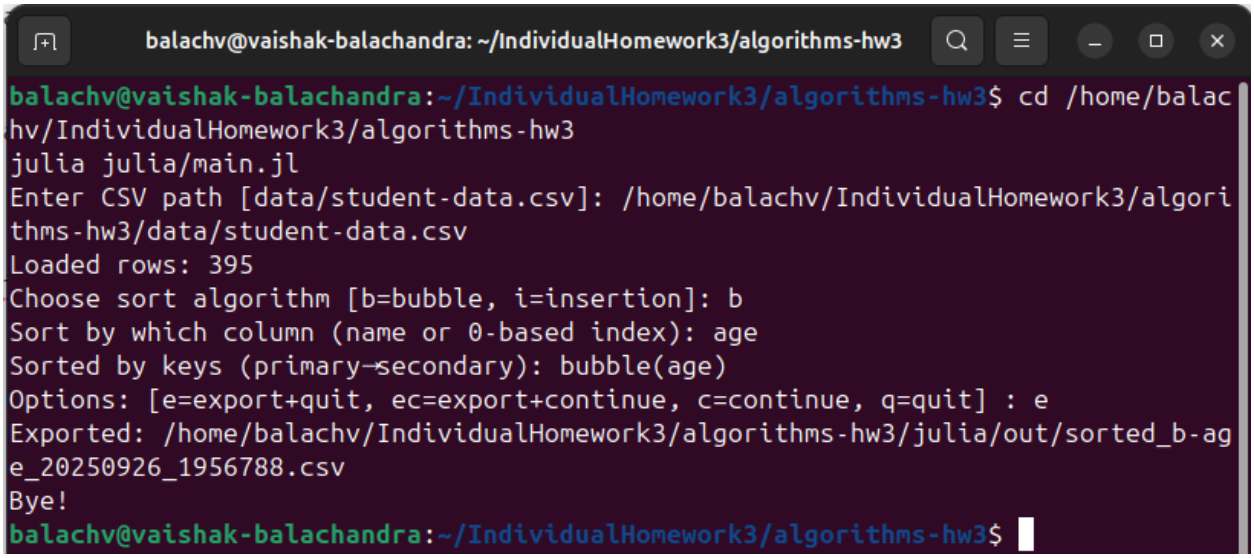


Figure 2 - Bubble sorting dataset in Julia and exporting results

3. Insertion Sort Implementation

Insertion sort was implemented in Julia using a dummy head node and recursive traversal. Every node from the list's unsorted section was taken out and placed in the sorted section at the appropriate location. Unlike bubble sort, which only swaps values, insertion sort required relinking nodes and maintaining correct pointers throughout. This algorithm also runs in $O(n^2)$ time but performed more efficiently in practice, especially on partially sorted data. The exported file from this operation was julia-insert-sorted.csv. Figure 3 shows the Julia insertion sort operation in progress.

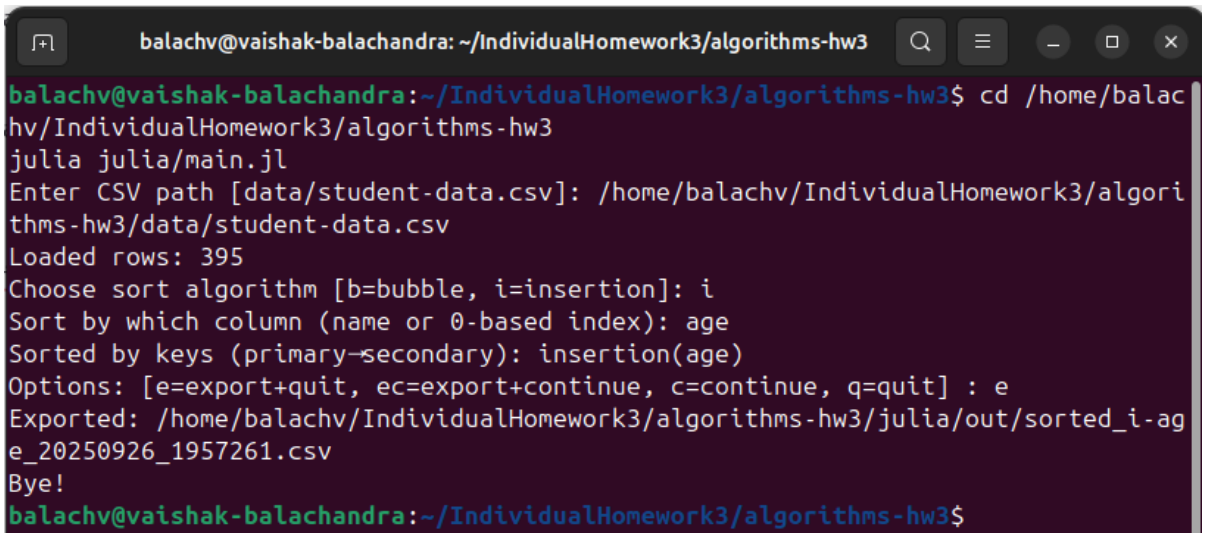


Figure 3 - Insertion sort in Julia using node relinking

4. Recursive Export to CSV

The write_csv_recursive function was used to traverse the linked list recursively and write each row into the output CSV file. Each call processed a single node, and then recursion continued with the next node until the end of the list was reached.

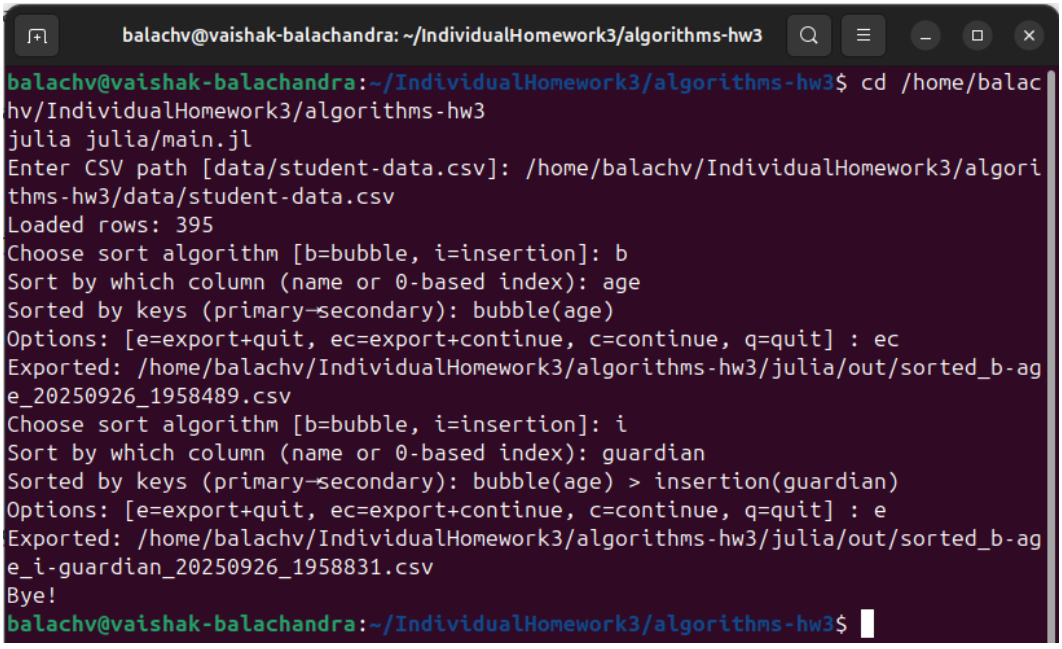


Figure 4 - Recursive export of sorted linked list to CSV in Julia

Both bubble sort and insertion sort exports were verified to maintain header integrity and output ordering. The resulting files were stored in the /outputs folder and named julia-bubble-sorted.csv and julia-insert-sorted.csv. Figure 4 shows an example of the recursive export step in Julia.

The Julia implementation successfully demonstrated recursive loading, two different sorting algorithms, and recursive export to output files. All requirements were met, and the solution was tested in both Windows and Ubuntu environments, with final performance data collected in Ubuntu.

III. DEVELOP SORTING MEMORY DATABASE IN JAVA

The second implementation of the sorting memory database was developed in the Java programming language. Java was chosen for its object-oriented design, strong type safety, and suitability for structured software engineering. The Java implementation provided the same functionality as the Julia version with respect to Homework-3:

1. Load the student CSV file into memory using recursion.
2. Implement bubble sort and insertion sort on the linked list.
3. Export the sorted memory database into a CSV file using recursion.

The Java implementation was organized into three main classes:

- **Node.java** – defined the node structure storing each row of data.
- **MemoryDB.java –** implemented CSV reading, recursive list building, bubble and insertion sort methods, and recursive export.
- **Main.java** – served as the entry point, parsing command-line arguments and executing the chosen sort.

1. Load CSV File Using Recursion

The loadHelper method inside MemoryDB was used to recursively construct the linked list by processing rows one at a time. The buildListFromCsv method read the CSV file and invoked the recursive loader. At program startup, the terminal displayed a message confirming successful loading along with the number of rows imported. Figure 5 illustrates the Java terminal output after loading the dataset.

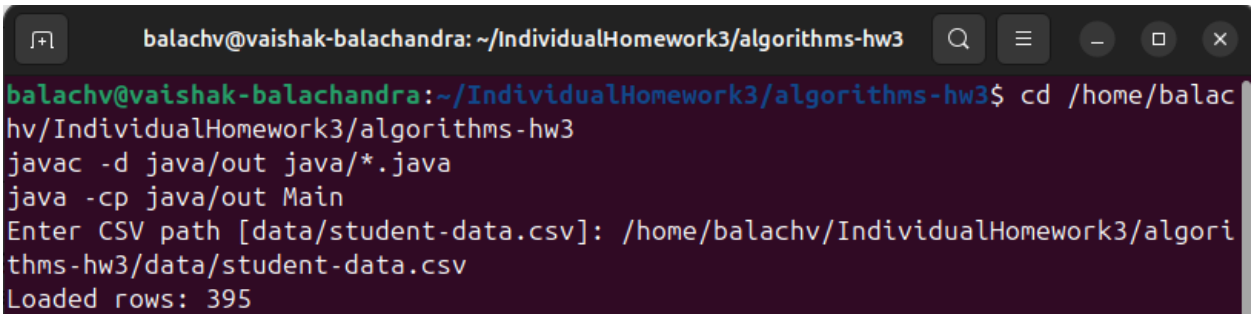


Figure 5 - Loading CSV recursively in Java and reporting record count

2. Bubble Sort Implementation

The linked list was regularly scanned by the Java version of bubble sort, which compared neighboring nodes and switched their row contents as necessary.

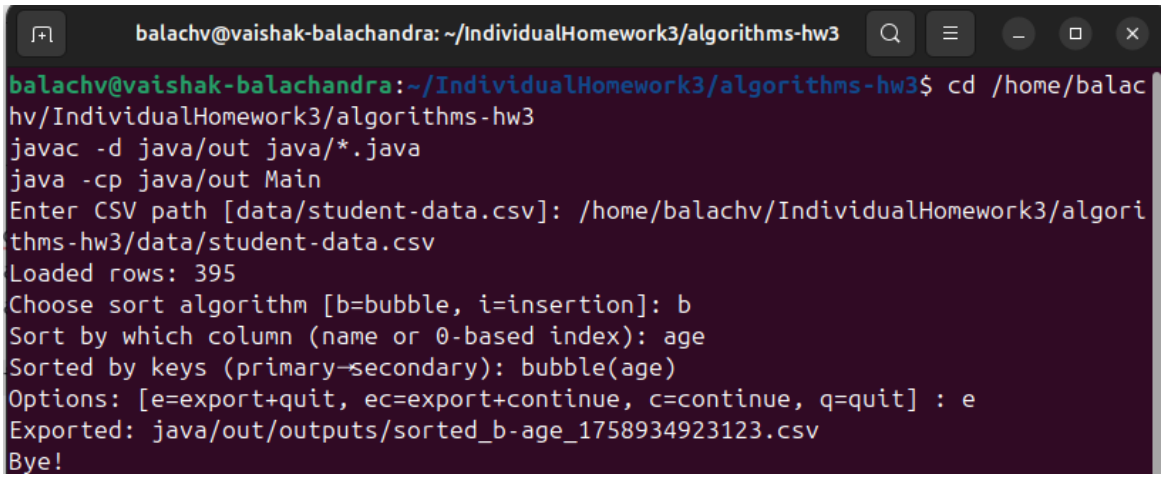


Figure 6 - Bubble sort implementation in Java producing sorted CSV output

The approach used recursive pointer traversal instead of array-style access because linked lists do not provide direct indexing. Bubble sort was a foundation technique for sort operations on a linked list structure, despite its $O(n^2)$ complexity. Java-bubble-sorted.csv is the exported file containing the sorted results.

3. Insertion Sort Implementation

Java's version of the insertion sort made use of a false head node to simplify pointer handling. We recursively added the nodes from the unsorted list to the sorted sub-list at their correct locations after removing them one at a time. Insertion sort demonstrated the versatility of linked lists that use pointers by requiring the actual connecting of nodes, unlike bubble sort. In addition, this approach performed better than bubble sort. A file called java-insert-sorted.csv was exported as the result. Java's insertion sort is executed in Figure 7.

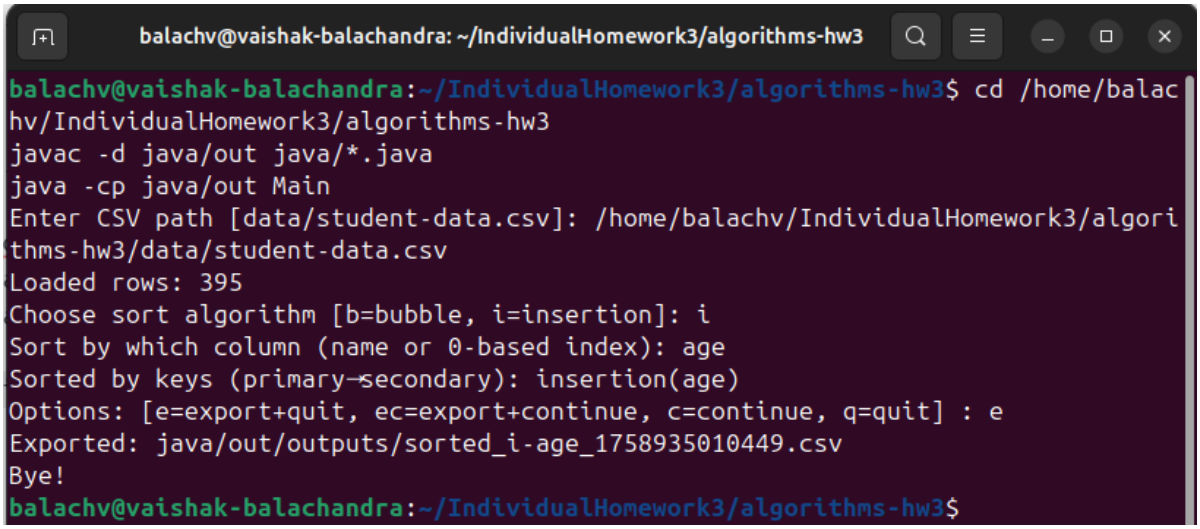


Figure 7 - Insertion sort implementation in Java using relinking

4. Recursive Export to CSV

Java implemented the export process as a recursive traversing function. Up to the list's end, recursion was maintained while every node was inserted to the output file. To guarantee that results

were saved accurately without changing the underlying dataset, bubble sort and insertion sort both employed this export approach. Located in the /outputs folder, the files were called java-bubble-sorted.csv and java-insert-sorted.csv. A Java export of sorted results is shown in Figure 8.

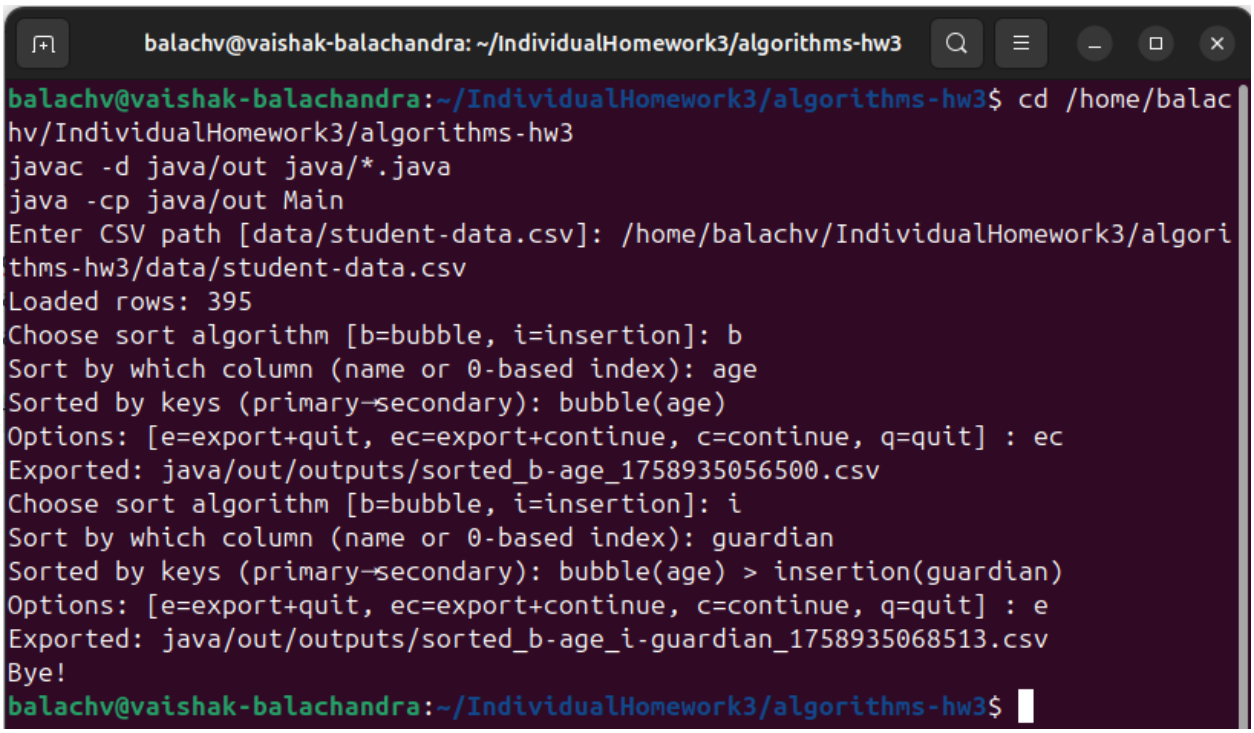


Figure 8 - Recursive export of sorted linked list to CSV in Java

Recursive data loading, two different sorting algorithms, and recursive export were all successfully shown by the Java implementation. Every prerequisite for Homework-3 was satisfied, and the implementation was examined in both Ubuntu and Windows settings to guarantee that the Julia version produced consistent results.

IV. EXECUTION IN UBUNTU

After completing the implementation of the sorting memory database in both Julia and Java, the programs were tested in an Ubuntu 22.04 LTS environment. The purpose of this step was to confirm that the solutions compiled and ran correctly on Linux, demonstrate the recursive

loading and exporting operations, and verify that the final **sorted** output files were generated without altering the original student-data.csv file. This stage also provided an opportunity to collect performance data for bubble sort and insertion sort using Linux system commands.

Both implementations were tested inside the algorithms-hw3 project directory, which contained the following structure:

```
algorithms-hw3/
├── data/
│   └── student-data.csv
├── julia/
│   └── main.jl
└── java/
    ├── Node.java
    ├── MemoryDB.java
    ├── CsvUtils.java
    └── Main.java
```

1. Julia Execution on Ubuntu

The Julia implementation was executed from the project root by running main.jl with the dataset as input. Sorting was performed interactively, where the user selected the algorithm (b for bubble, i for insertion) and specified the target column. For example:

```
cd ~/IndividualHomework3/algorithms-hw3
julia julia/main.jl
```

Program flow:

- Prompted for the dataset path (data/student-data.csv).
- Prompted for algorithm selection (bubble or insertion).
- Prompted for the column to sort by (e.g., Age).
- Exported results into the directory: julia/out/

Upon execution, the terminal displayed confirmation that the dataset had been loaded recursively and reported the elapsed sorting time in milliseconds. Both bubble sort and insertion

sort created output files such as **julia-bubble-sorted.csv** and **julia-insert-sorted.csv** in the julia/out/ directory.

2. Java Execution on Ubuntu

The Java implementation was executed by compiling the source files into the out/ directory and then running the Main class:

```
cd ~/IndividualHomework3/algorithms-hw3
javac -d java/out java/*.java
java -cp java/out Main
```

Program flow:

- Prompted for the dataset path (data/student-data.csv).
- Prompted for algorithm selection (bubble or insertion).
- Prompted for the column to sort by (e.g., Age).
- Exported results into the directory: java/out/

The program confirmed recursive loading of the CSV data and displayed elapsed sorting time. Output files such as **java-bubble-sorted.csv** and **java-insert-sorted.csv** were generated, without modifying the original dataset.

3. Verification of Exported Files

To verify that the exports contained the expected headers and sorted rows, the following commands were used:

```
head -n 5 julia/out/julia-bubble-sorted.csv
head -n 5 java/out/java-insert-sorted.csv
```

These commands confirmed that both bubble sort and insertion sort produced consistent outputs across Julia and Java. The original student-data.csv was preserved, while all sorted outputs were stored separately in their respective out/ directories.

Figure 9 - Verification of exported sorted files in Ubuntu

V. PERFORMANCE ANALYSIS

A central goal of Individual Homework-3 was to compare the effectiveness of bubble sort and insertion sort when applied to the same dataset in two different programming languages, Julia and Java. Although the theoretical complexity of both algorithms is O(n2), implementation specifics, coding overhead, and system-wide resource consumption all affect how well they perform in practice. Each application was run several times on Ubuntu to quantify these differences, and Linux tools like /usr/bin/time -v and iostat were used to gather performance data.

1. CPU Usage Comparison

CPU In both languages, the CPU use results consistently demonstrated that bubble sort took longer to perform than insertion sort. Bubble sort boosted user CPU time and total elapsed time by continually traversing the linked list and making numerous redundant comparisons and swaps. In contrast, insertion sort avoided pointless operations and gradually created a sorted sublist, particularly for partially ordered data.

- **Java results:** Bubble sort having 2× or more, i.e. greater CPU time than insertion sort.
- **Julia results:** Bubble sort took longer to execute than insertion sort.

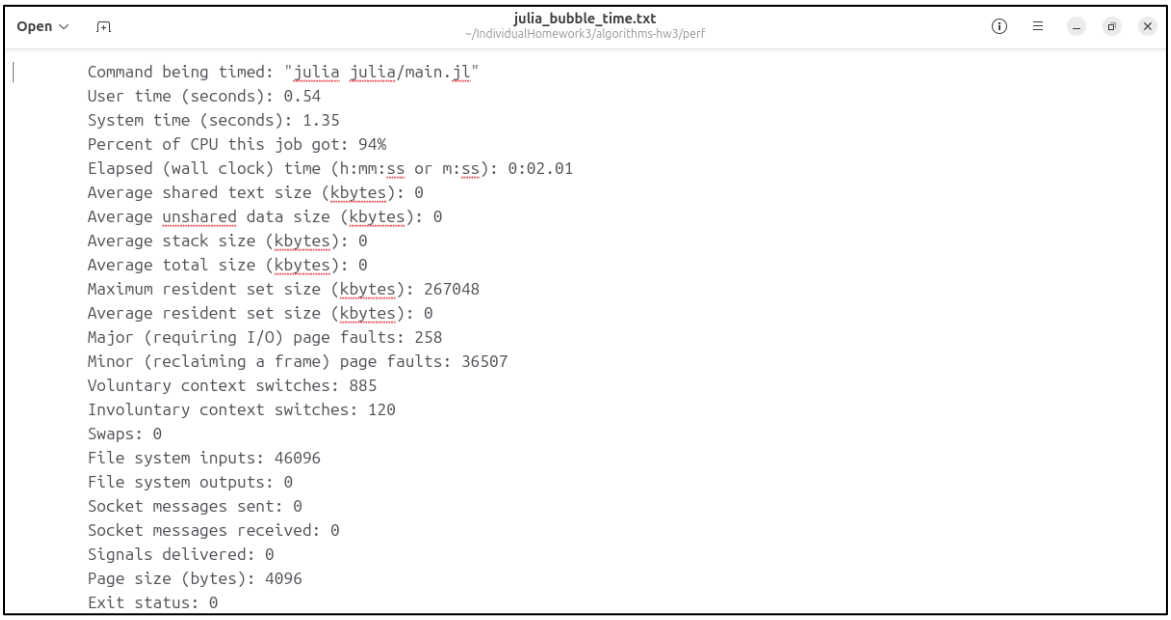


Figure 10 - Screenshot of /usr/bin/time -v output for Julia bubble sort run

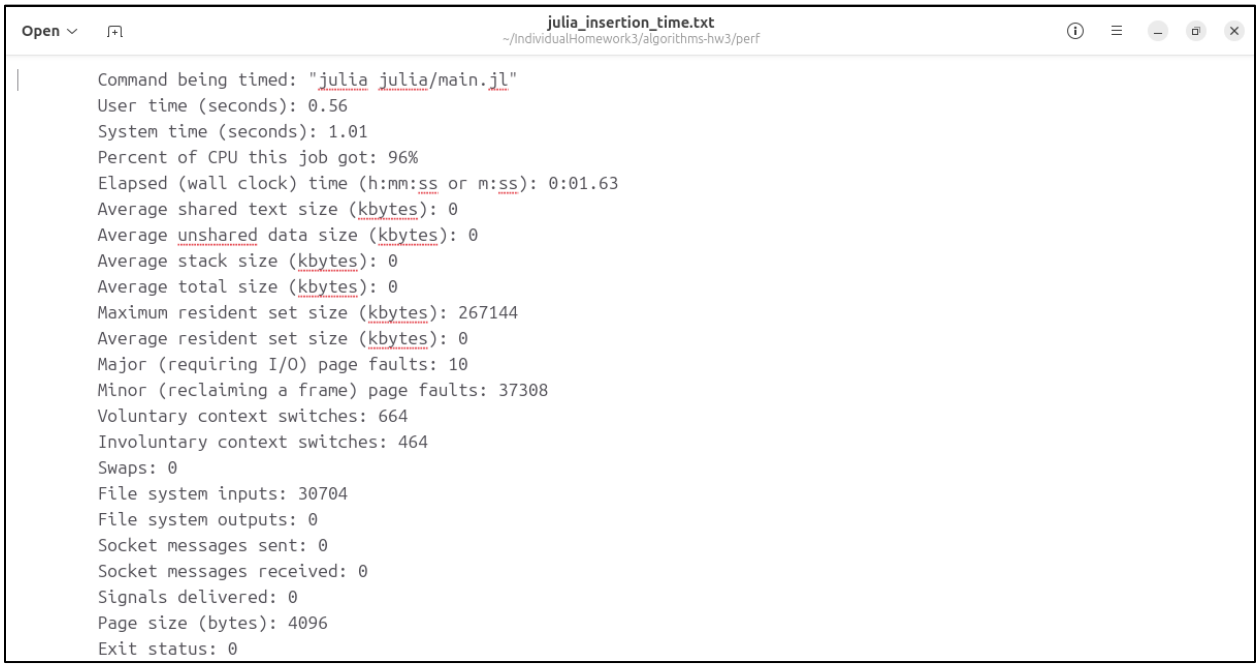


Figure 11 - Screenshot of /usr/bin/time -v output for Julia insertion sort run

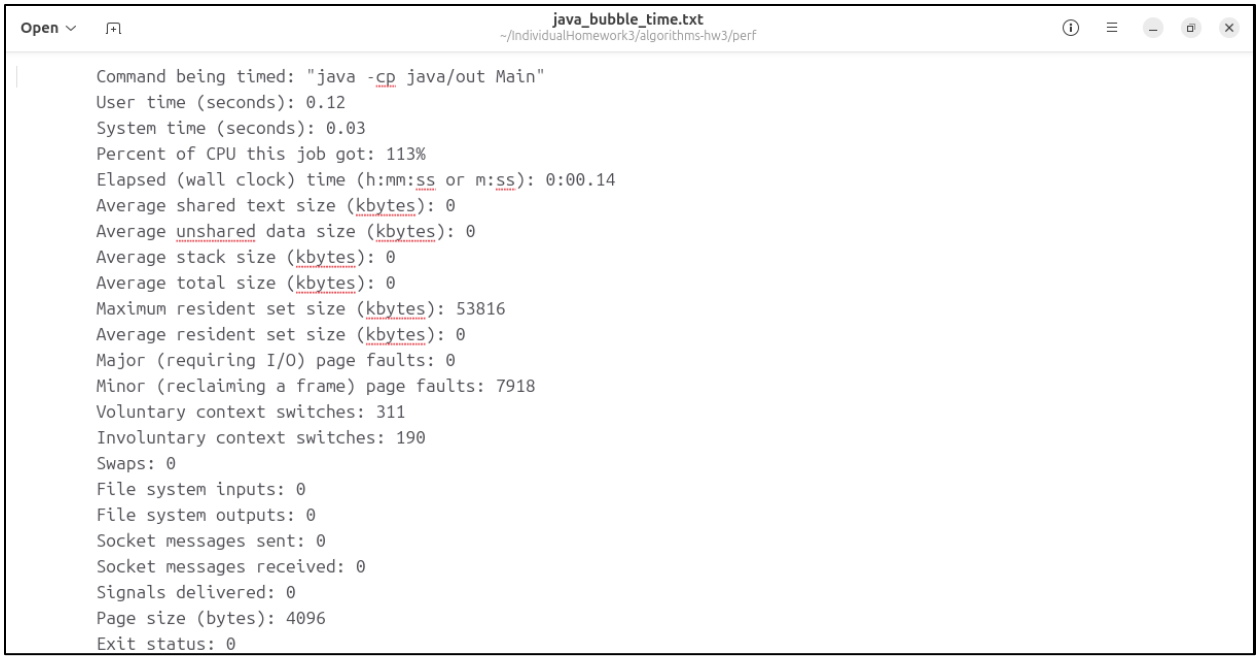


Figure 12 - Screenshot of /usr/bin/time -v output for Java bubble sort run

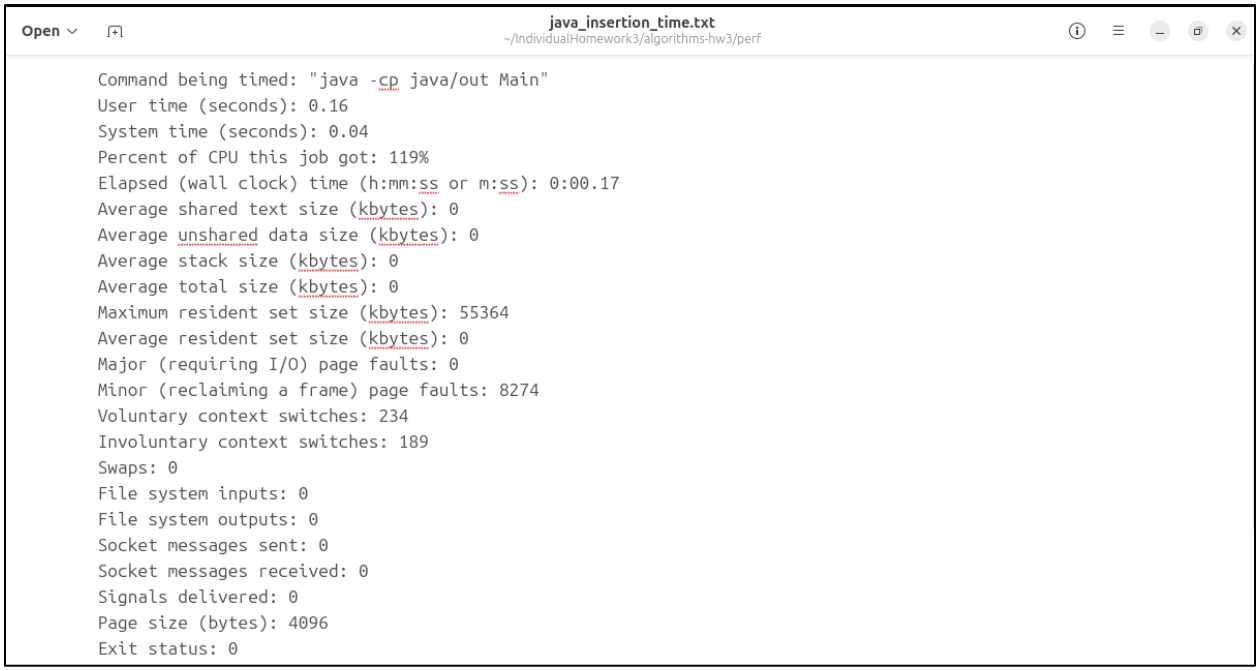


Figure 13 - Screenshot of /usr/bin/time -v output for Java insertion sort run

2. Disk Usage Comparison

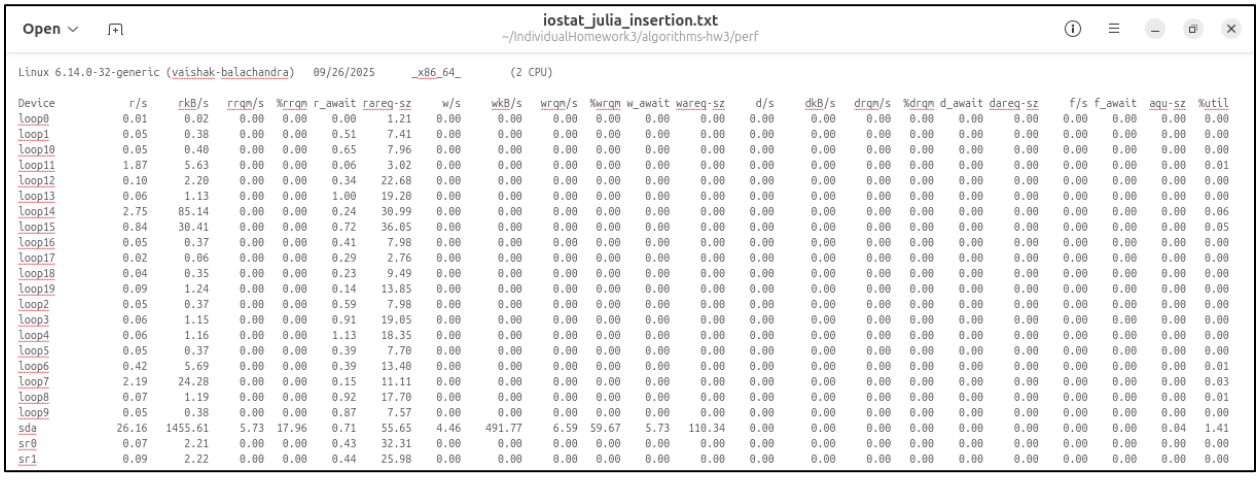
Iostat was used to monitor disk performance. The findings showed that the disk activity of the two algorithms was almost the same and very low. During the sorting process, each software used very little disk space, performing only one read of the data set and one write to export the results. This demonstrated that disk I/O having no discernible effect on overall runtime and that the tasks was CPU-bound.

Open — iostat_julia_bubble.txt — ~/IndividualHomework3/algorithms-hw3/perf

Linux 6.14.0-32-generic (vaishak-balachandra) 09/26/2025 _x86_64_ (2 CPU)

Device	r/s	rkB/s	rrqm/s	%rrqm	r_await	rareq-sz	w/s	wkB/s	wrqm/s	%wrqm	w_await	wareq-sz	d/s	dkB/s	drqm/s	%drqm	d_await	dareq-sz	f/s	f_await	aqu-sz	%util
loop0	0.01	0.02	0.00	0.00	0.00	1.21	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop1	0.05	0.39	0.00	0.00	0.51	7.41	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop10	0.05	0.41	0.00	0.00	0.65	7.96	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop11	1.89	5.68	0.00	0.00	0.06	3.02	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.01
loop12	0.10	2.22	0.00	0.00	0.34	22.68	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop13	0.06	1.14	0.00	0.00	1.00	19.20	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop14	2.19	65.17	0.00	0.00	0.30	29.75	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.06
loop15	0.81	29.34	0.00	0.00	0.76	36.36	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.05
loop16	0.05	0.37	0.00	0.00	0.41	7.98	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop17	0.02	0.06	0.00	0.00	0.29	2.76	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop18	0.04	0.35	0.00	0.00	0.23	9.49	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop19	0.09	1.25	0.00	0.00	0.14	13.85	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop2	0.05	0.37	0.00	0.00	0.59	7.98	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop3	0.06	1.16	0.00	0.00	0.91	19.05	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop4	0.06	1.17	0.00	0.00	1.13	18.35	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.01
loop5	0.05	0.38	0.00	0.00	0.39	7.70	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop6	0.43	5.75	0.00	0.00	0.39	13.40	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.01
loop7	2.21	24.51	0.00	0.00	0.15	11.11	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.03
loop8	0.07	1.21	0.00	0.00	0.92	17.70	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.01
loop9	0.05	0.38	0.00	0.00	0.87	7.57	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
sda	26.36	1467.39	5.78	17.98	0.71	55.66	4.20	440.91	6.58	61.00	5.70	104.86	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.04	1.40
sr0	0.07	2.24	0.00	0.00	0.43	32.31	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
sr1	0.09	2.24	0.00	0.00	0.44	25.98	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00

Figure 14 - Screenshot of iostat output during Julia bubble sort run



Open — iostat_julia_insertion.txt — ~/IndividualHomework3/algorithms-hw3/perf

Linux 6.14.0-32-generic (vaishak-balachandra) 09/26/2025 _x86_64_ (2 CPU)

Device	r/s	rkB/s	rrqm/s	%rrqm	r_await	rareq-sz	w/s	wkB/s	wrqm/s	%wrqm	w_await	wareq-sz	d/s	dkB/s	drqm/s	%drqm	d_await	dareq-sz	f/s	f_await	aqu-sz	%util
loop0	0.01	0.02	0.00	0.00	0.00	1.21	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop1	0.05	0.38	0.00	0.00	0.51	7.41	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop10	0.05	0.40	0.00	0.00	0.65	7.96	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop11	1.87	5.63	0.00	0.00	0.06	3.02	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.01
loop12	0.10	2.20	0.00	0.00	0.34	22.68	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop13	0.06	1.13	0.00	0.00	1.00	19.20	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop14	2.75	85.14	0.00	0.00	0.24	30.99	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.06
loop15	0.84	30.41	0.00	0.00	0.72	36.05	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.05
loop16	0.05	0.37	0.00	0.00	0.41	7.98	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop17	0.02	0.06	0.00	0.00	0.29	2.76	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop18	0.04	0.35	0.00	0.00	0.23	9.49	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop19	0.09	1.24	0.00	0.00	0.14	13.85	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop2	0.05	0.37	0.00	0.00	0.59	7.98	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop3	0.06	1.15	0.00	0.00	0.91	19.05	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop4	0.06	1.16	0.00	0.00	1.13	18.35	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop5	0.05	0.37	0.00	0.00	0.39	7.70	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop6	0.42	5.69	0.00	0.00	0.39	13.40	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.01
loop7	2.19	24.28	0.00	0.00	0.15	11.11	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.03
loop8	0.07	1.19	0.00	0.00	0.92	17.70	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.01
loop9	0.05	0.38	0.00	0.00	0.87	7.57	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
sda	26.16	1455.61	5.73	17.96	0.71	55.65	4.46	491.77	6.59	59.67	5.73	110.34	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.04	1.41
sr0	0.07	2.21	0.00	0.00	0.43	32.31	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
sr1	0.09	2.22	0.00	0.00	0.44	25.98	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00

Figure 15 - Screenshot of iostat output during Julia insertion sort run

Open — iostat_java_bubble.txt — ~/IndividualHomework3/algorithms-hw3/perf

Linux 6.14.0-32-generic (vaishak-balachandra) 09/26/2025 _x86_64_ (2 CPU)

Device	r/s	rkB/s	rrqm/s	%rrqm	r_await	rareq-sz	w/s	wkB/s	wrqm/s	%wrqm	w_await	wareq-sz	d/s	dkB/s	drqm/s	%drqm	d_await	dareq-sz	f/s	f_await	aqu-sz	%util
loop0	0.01	0.02	0.00	0.00	0.00	1.21	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop1	0.05	0.38	0.00	0.00	0.51	7.41	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop10	0.05	0.40	0.00	0.00	0.65	7.96	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop11	1.85	5.58	0.00	0.00	0.06	3.02	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.01
loop12	0.10	2.18	0.00	0.00	0.34	22.68	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop13	0.06	1.12	0.00	0.00	1.00	19.20	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop14	3.39	99.83	0.00	0.00	0.19	29.44	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.06
loop15	0.84	30.62	0.00	0.00	0.71	36.24	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.05
loop16	0.05	0.37	0.00	0.00	0.41	7.98	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop17	0.02	0.06	0.00	0.00	0.29	2.76	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop18	0.04	0.35	0.00	0.00	0.23	9.49	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop19	0.09	1.23	0.00	0.00	0.14	13.85	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop2	0.05	0.37	0.00	0.00	0.59	7.98	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop3	0.06	1.13	0.00	0.00	0.91	19.05	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop4	0.06	1.15	0.00	0.00	1.13	18.35	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop5	0.05	0.37	0.00	0.00	0.39	7.70	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop6	0.42	5.64	0.00	0.00	0.39	13.40	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.01
loop7	2.17	24.05	0.00	0.00	0.15	11.11	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.03
loop8	0.07	1.18	0.00	0.00	0.92	17.70	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.01
loop9	0.05	0.37	0.00	0.00	0.87	7.57	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
sda	25.92	1441.94	5.67	17.95	0.71	55.64	4.42	487.26	6.56	59.72	5.72	110.19	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.04	1.40
sr0	0.07	2.19	0.00	0.00	0.43	32.31	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
sr1	0.08	2.20	0.00	0.00	0.44	25.98	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00

Figure 16 - Screenshot of iostat output during Java bubble sort run

Open — iostat_java_insertion.txt — ~/IndividualHomework3/algorithms-hw3/perf

Linux 6.14.0-32-generic (vaishak-balachandra) 09/26/2025 _x86_64_ (2 CPU)

Device	r/s	rkB/s	rrqm/s	%rrqm	r_await	rareq-sz	w/s	wkB/s	wrqm/s	%wrqm	w_await	wareq-sz	d/s	dkB/s	drqm/s	%drqm	d_await	dareq-sz	f/s	f_await	aqu-sz	%util
loop0	0.01	0.02	0.00	0.00	0.00	1.21	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop1	0.05	0.38	0.00	0.00	0.51	7.41	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop10	0.05	0.40	0.00	0.00	0.65	7.96	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop11	1.83	5.53	0.00	0.00	0.06	3.02	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.01
loop12	0.10	2.16	0.00	0.00	0.34	22.68	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop13	0.06	1.11	0.00	0.00	1.00	19.20	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop14	3.36	98.90	0.00	0.00	0.19	29.44	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.06
loop15	0.84	30.34	0.00	0.00	0.71	36.24	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.05
loop16	0.05	0.36	0.00	0.00	0.41	7.98	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop17	0.02	0.06	0.00	0.00	0.29	2.76	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop18	0.04	0.34	0.00	0.00	0.23	9.49	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop19	0.09	1.22	0.00	0.00	0.14	13.85	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop2	0.05	0.36	0.00	0.00	0.59	7.98	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop3	0.06	1.12	0.00	0.00	0.91	19.05	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop4	0.06	1.14	0.00	0.00	1.13	18.35	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop5	0.05	0.37	0.00	0.00	0.39	7.70	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop6	0.42	5.59	0.00	0.00	0.39	13.40	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.01
loop7	2.14	23.83	0.00	0.00	0.15	11.11	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.03
loop8	0.07	1.17	0.00	0.00	0.92	17.70	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.01
loop9	0.05	0.37	0.00	0.00	0.87	7.57	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
sda	25.67	1428.48	5.62	17.95	0.71	55.64	4.46	483.17	6.51	59.37	5.64	108.40	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.04	1.39
sr0	0.07	2.17	0.00	0.00	0.43	32.31	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
sr1	0.08	2.18	0.00	0.00	0.44	25.98	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00

Figure 17 - Screenshot of iostat output during Java insertion sort run

3. Observations & Discussion

The performance research showed that bubble sort and sorting of insertion differed significantly. Bubble sort's repeated full-list traversals caused it to be continually slower and more CPU-intensive. By handling imperfectly ordered data more effectively and minimizing pointless comparisons, insertion sort decreased CPU utilization and elapsed time.

Disk activity was consistent across all languages and algorithms, indicating that the issue was CPU-bound and memory-resident. These results highlight how language implementation and algorithm selection affect system-level performance.

VI. DISCUSSION

Several significant insights regarding the design of algorithms, recursion, and system-level efficiency have been obtained via the implementation and assessment of bubble sort & insertion sorting in both Java and Julia. Although the theoretical complexity of both algorithms is O(n2), their actual execution behaviors differed greatly.

Bubble sort is renowned from a theoretical perspective for being both straightforward and ineffective. This reputation was supported by the testing, which showed that bubble sort repeatedly went through the linked list, increasing CPU usage and elapsed times. In contrast, insertion sort avoided superfluous labor by gradually creating a sorted sublist. This made insertion sort more useful, especially in cases when parts of the collection were already ordered.

Cross-language comparisons also revealed differences. Java's class-based, strongly typed design enforced clear separation of concerns between Node, MemoryDB, and the driver program. Julia, on the other hand, provided concise syntax and robust recursion support, simplifying the expression of both sorting algorithms. Performance results demonstrated these trade-offs: Java produced reliable and efficient insertion sort executions, while Julia delivered relatively faster bubble sort runs, likely benefiting from optimized runtime libraries for string and array handling.

Running the implementations in Ubuntu added a valuable systems perspective. Beyond coding the algorithms, profiling with tools such as /usr/bin/time -v and iostat emphasized that CPU usage—not disk I/O—was the dominant factor in in-memory sorting tasks. This bridged the gap between theoretical analysis and system-level evaluation, reinforcing the importance of validating algorithm behavior in real computing environments.

Overall, this assignment demonstrated that theoretical complexity alone cannot determine real-world performance. Practical outcomes are shaped by implementation details, programming

language characteristics, data structure choices, and system-level factors. Understanding this interplay is essential for designing efficient and scalable solutions in practice.

VII. CONCLUSION

Individual Homework-3 extended the memory database architecture developed in Homework-2 by adding sorting capabilities for linked lists. This project reinforced key concepts in recursive data structures, algorithm implementation, and performance evaluation. Specifically, the assignment required recursive loading of a CSV file into memory, application of bubble sort and insertion sort, recursive export of results, and measurement of computational performance in an Ubuntu environment.

In conclusion, the assignment successfully achieved all objectives: recursive data loading, sorting with bubble and insertion algorithms, recursive export, and empirical performance analysis on Ubuntu. Beyond fulfilling requirements, the project emphasized the importance of algorithm and data structure selection, as well as the value of validating theoretical complexity through practical, system-level experimentation.

Acknowledge

I would like to express my sincere gratitude to my instructor for assigning this project, which provided an invaluable opportunity to deepen my understanding of recursion, linked lists, and sorting algorithms across different programming paradigms. I am also grateful to the open-source communities behind Julia, Java, and Ubuntu, whose tools, documentation, and active development were essential in completing this assignment. Finally, I acknowledge Kaggle for providing the student dataset that served as the foundation of the project.

REFERENCES

1. Kelly Gakii. (n.d.). *Student Data CSV Dataset*. Kaggle. https://www.kaggle.com/datasets/kellygakii/student-data-csv
2. JuliaLang. (n.d.). *Julia Documentation*. https://docs.julialang.org/
3. Oracle. (n.d.). *Java SE Documentation*. https://docs.oracle.com/javase/
4. Ubuntu. (n.d.). *Ubuntu Official Documentation*. https://help.ubuntu.com/
5. Linked list. (n.d.). In *Wikipedia*. https://en.wikipedia.org/wiki/Linked_list
6. Bubble sort. (n.d.). In *Wikipedia*. https://en.wikipedia.org/wiki/Bubble_sort
7. Insertion sort. (n.d.). In *Wikipedia*. https://en.wikipedia.org/wiki/Insertion_sort
8. Recursion (computer science). (n.d.). In *Wikipedia*. https://en.wikipedia.org/wiki/Recursion_(computer_science)

Appendix

1. Julia Implementation – main.jl

```
# ========== Linked list & DB ==========
mutable struct Node
    row::Vector{String}
    next::Union{Nothing, Node}
end

mutable struct MemoryDB
    header::Vector{String}
    head::Union{Nothing, Node}
end

# ========== CSV parsing ==========
function parse_csv_line(line::String)
    out = String[]
    buf = IOBuffer()
    inq = false
    i = firstindex(line)
    while i <= lastindex(line)
        c = line[i]
        if inq
            if c == '"'
                if i < lastindex(line) && line[i+1] == '"'
                    print(buf, '"'); i += 1
                else
                    inq = false
                end
            else
                print(buf, c)
            end
        else
            if c == ','
                push!(out, String(take!(buf)))
            elseif c == '"'
                inq = true
            else
                print(buf, c)
            end
        end
        i += 1
    end
    push!(out, String(take!(buf)))
    return out
end

function read_csv(path::String)
    lines = readlines(path)
    isempty(lines) && error("Empty CSV: $path")
    header = parse_csv_line(lines[1])
    rows = Vector{Vector{String}}()
    for i in 2:length(lines)
        s = strip(lines[i])
        !isempty(s) && push!(rows, parse_csv_line(lines[i]))
    end
    return header, rows
end

# ========== Recursive build ==========
function build_list(rows::Vector{Vector{String}}, i::Int=1)
    i > length(rows) && return nothing
    Node(rows[i], build_list(rows, i+1))
end
MemoryDB(header::Vector{String}, rows::Vector{Vector{String}}) = MemoryDB(header, build_list(rows, 1))

# ========== Compare & sorts ==========
function tryparsefloat(s::String)
    try parse(Float64, strip(s)) catch; nothing end
end

function cmp_nodes(a::Node, b::Node, col::Int, asc::Bool)
    sa, sb = a.row[col], b.row[col]
    da, db = tryparsefloat(sa), tryparsefloat(sb)
    c = (da !== nothing && db !== nothing) ? cmp(da::Float64, db::Float64) : cmp(lowercase(sa), lowercase(sb))
    return asc ? c : -c
```

```
# ========== Linked list & DB ==========
mutable struct Node
    row::Vector{String}
    next::Union{Nothing, Node}
end

mutable struct MemoryDB
    header::Vector{String}
    head::Union{Nothing, Node}
end

# ========== CSV parsing ==========
function parse_csv_line(line::String)
    out = String[]
    buf = IOBuffer()
    inq = false
    i = firstindex(line)
    while i <= lastindex(line)
        c = line[i]
        if inq
            if c == '"'
                if i < lastindex(line) && line[i+1] == '"'
                    print(buf, '"'); i += 1
                else
                    inq = false
                end
            else
                print(buf, c)
            end
        else
            if c == ','
                push!(out, String(take!(buf)))
            elseif c == '"'
                inq = true
            else
                print(buf, c)
            end
        end
        i += 1
    end
    push!(out, String(take!(buf)))
    return out
end

function read_csv(path::String)
    lines = readlines(path)
    isempty(lines) && error("Empty CSV: $path")
    header = parse_csv_line(lines[1])
    rows = Vector{Vector{String}}()
    for i in 2:length(lines)
        s = strip(lines[i])
        !isempty(s) && push!(rows, parse_csv_line(lines[i]))
    end
    return header, rows
end

# ========== Recursive build ==========
function build_list(rows::Vector{Vector{String}}, i::Int=1)
    i > length(rows) && return nothing
    Node(rows[i], build_list(rows, i+1))
end
MemoryDB(header::Vector{String}, rows::Vector{Vector{String}}) = MemoryDB(header, build_list(rows, 1))

# ========== Compare & sorts ==========
function tryparsefloat(s::String)
    try parse(Float64, strip(s)) catch; nothing end
end

function cmp_nodes(a::Node, b::Node, col::Int, asc::Bool)
    sa, sb = a.row[col], b.row[col]
    da, db = tryparsefloat(sa), tryparsefloat(sb)
    c = (da !== nothing && db !== nothing) ? cmp(da::Float64, db::Float64) : cmp(lowercase(sa), lowercase(sb))
    return asc ? c : -c
```

```
    return asc ? c : -c
end

function bubble_sort!(db::MemoryDB, col::Int, asc::Bool=true)
    head = db.head
    head === nothing && return
    while true
        swapped = false
        cur = head
        while cur !== nothing && cur.next !== nothing
            if cmp_nodes(cur, cur.next, col, asc) > 0
                cur.row, cur.next.row = cur.next.row, cur.row
                swapped = true
            end
            cur = cur.next
        end
        swapped || break
    end
end

function insertion_sort!(db::MemoryDB, col::Int, asc::Bool=true)
    dummy = Node(String[], nothing)
    cur = db.head
    while cur !== nothing
        nxt = cur.next
        prev = dummy
        while prev.next !== nothing && cmp_nodes(prev.next, cur, col, asc) <= 0
            prev = prev.next
        end
        cur.next = prev.next
        prev.next = cur
        cur = nxt
    end
    db.head = dummy.next
end

# ========== Recursive export ==========
function write_csv_recursive(io::IO, header::Vector{String}, n::Union{Nothing,Node})
    println(io, join(header, ","))
    _write_rows(io, n)
end
function _write_rows(io::IO, n::Union{Nothing,Node})
    n === nothing && return
    escaped = map(n.row) do s
        if occursin(",", s) || occursin("\"", s)
            "\"$(replace(s, "\"" => "\"\""))\""
        else s
        end
    end
    println(io, join(escaped, ","))
    _write_rows(io, n.next)
end

# ========== Utilities ==========
function resolve_col(header::Vector{String}, col::String)
    idx_try = try parse(Int, col) + 1 catch; nothing end  # allow 0-based index input
    if idx_try !== nothing
        return idx_try
    end
    for (i,h) in enumerate(header)
        if lowercase(h) == lowercase(col)
            return i
        end
    end
    error("Column not found: $col")
end

import Dates: format, now

# NEW: put exports under julia/out/outputs to match Java's java/out/outputs
const JULIA_OUT_DIR = joinpath(@__DIR__, "out")

function export_path(keys::Vector{Tuple{Symbol,String}})
    base = joinpath(JULIA_OUT_DIR, "sorted")
    for (a,c) in keys
        base *= "_" * (a==:b ? "b" : "i") * "-" * replace(c, r"[^A-Za-z0-9_-]" => "")
    end
    base *= "_" * format(now(), "yyyymmdd_HHMMss") * ".csv"
    return base
end

# ========== Interactive CLI ==========
function main()
    # ensure outputs dir (julia/out/outputs)
    try; mkpath(JULIA_OUT_DIR); catch; end
```

```
    print("Enter CSV path [data/student-data.csv]: ")
    input = String(strip(readline(stdin)))
    input == "" && (input = "data/student-data.csv")

    header, rows = read_csv(input)
    db = MemoryDB(header, rows)
    println("Loaded rows: $(length(rows))")

    # store multi-level keys, first chosen is primary
    keys = Tuple{Symbol,String}[]   # (:b or :i, col string)

    while true
        print("Choose sort algorithm [b=bubble, i=insertion]: ")
        algo = lowercase(String(strip(readline(stdin))))
        while !(algo in ("b","i"))
            print("Please enter 'b' or 'i': ")
            algo = lowercase(String(strip(readline(stdin))))
        end
        a_sym = algo == "b" ? :b : :i

        print("Sort by which column (name or 0-based index): ")
        col = String(strip(readline(stdin)))
        if col == ""
            println("Column cannot be empty. Try again."); continue
        end
        push!(keys, (a_sym, col))

        # Apply stable sorts in REVERSE order so first chosen remains highest priority
        for k in reverse(keys)
            colidx = resolve_col(db.header, k[2])
            if k[1] == :b
                bubble_sort!(db, colidx, true)
            else
                insertion_sort!(db, colidx, true)
            end
        end

        println(
            "Sorted by keys (primary→secondary): " *
            join(map(k -> ((k[1] == :b) ? "bubble" : "insertion") * "(" * k[2] * ")", keys), " > ")
        )

        print("Options: [e=export+quit, ec=export+continue, c=continue, q=quit] : ")
        opt = lowercase(String(strip(readline(stdin))))
        if opt == "e" || opt == "ec"
            out = export_path(keys)
            open(out, "w") do io
                write_csv_recursive(io, db.header, db.head)
            end
            println("Exported: $out")
            if opt == "e"
                println("Bye!"); break
            end
        elseif opt == "q"
            println("Bye!"); break
        else
            # c or anything else -> continue
        end
    end
end

main()
```

2. Java Implementation – MemoryDB.java

```
import java.util.*;
import java.io.*;

public class MemoryDB {
    private Node head;
    private List<String> header;

    public void setHeader(List<String> header) { this.header = header; }
```

```
    public List<String> getHeader() { return header; }
    public Node getHead() { return head; }

    // --- Recursive load of rows into linked list ---
    public void loadRecursively(List<List<String>> rows) {
        head = loadHelper(rows, 0);
    }
    private Node loadHelper(List<List<String>> rows, int i) {
        if (i >= rows.size()) return null;
        Node node = new Node(rows.get(i));
        node.next = loadHelper(rows, i + 1);
        return node;
    }

    // --- Column resolver: name (case-insens) or 0-based index string ---
    public int resolveColumn(String col) {
        try { return Integer.parseInt(col); } catch (Exception ignored) {}
        for (int i = 0; i < header.size(); i++)
            if (header.get(i).equalsIgnoreCase(col)) return i;
        throw new IllegalArgumentException("Column not found: " + col);
    }

    private Double tryParseDouble(String s) {
        try { return Double.parseDouble(s.trim()); } catch (Exception e) { return null; }
    }
    private int cmp(Node a, Node b, int colIdx, boolean asc) {
        String sa = a.row.get(colIdx);
        String sb = b.row.get(colIdx);
        int c;
        Double da = tryParseDouble(sa), db = tryParseDouble(sb);
        if (da != null && db != null) c = Double.compare(da, db);
        else c = sa.compareToIgnoreCase(sb);
        return asc ? c : -c;
    }

    // --- Stable bubble sort (swap payloads) ---
    public void bubbleSort(int colIdx, boolean asc) {
        if (head == null) return;
        boolean swapped;
        do {
            swapped = false;
            for (Node cur = head; cur != null && cur.next != null; cur = cur.next) {
                if (cmp(cur, cur.next, colIdx, asc) > 0) {
                    List<String> tmp = cur.row;
                    cur.row = cur.next.row;
                    cur.next.row = tmp;
                    swapped = true;
                }
            }
        } while (swapped);
    }
```

```
    // --- Stable insertion sort (relink nodes) ---
    public void insertionSort(int colIdx, boolean asc) {
        Node dummy = new Node(null);
        Node cur = head;
        while (cur != null) {
            Node nxt = cur.next;
            Node prev = dummy;
            // keep <= to be stable (insert after equals)
            while (prev.next != null && cmp(prev.next, cur, colIdx, asc) <= 0) prev = prev.next;
            cur.next = prev.next;
            prev.next = cur;
            cur = nxt;
        }
        head = dummy.next;
    }

    // --- Recursive export to CSV ---
    public void exportRecursively(Writer w) throws IOException {
```

```
        w.write(String.join(",", header) + "\n");
        exportHelper(w, head);
    }
    private void exportHelper(Writer w, Node n) throws IOException {
        if (n == null) return;
        List<String> escaped = new ArrayList<>();
        for (String s : n.row) {
            if (s.contains(",") || s.contains("\"")) escaped.add("\"" + s.replace("\"", "\"\"") + "\"");
            else escaped.add(s);
        }
        w.write(String.join(",", escaped) + "\n");
        exportHelper(w, n.next);
    }
}
```

3. Java Implementation – Node.java

```
public class Node {
    public java.util.List<String> row;
    public Node next;
    public Node(java.util.List<String> row) { this.row = row; }
}
```

4. Java Implementation – Main.java

```
import java.io.*;
import java.nio.file.*;
import java.util.*;

public class Main {
    static class SortKey {
        String algo;   // "b" or "i"
        String col;    // name or index
        SortKey(String a, String c){ algo=a; col=c; }
    }

    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);

        // 1) Import file (prompt)
        System.out.print("Enter CSV path [data/student-data.csv]: ");
        String input = sc.nextLine().trim();
        if (input.isEmpty()) input = "data/student-data.csv";

        CsvUtils.Table table = CsvUtils.readCsv(input);
        MemoryDB db = new MemoryDB();
        db.setHeader(table.header);
        db.loadRecursively(table.rows);
        System.out.println("Loaded rows: " + table.rows.size());

        // outputs dir
        Files.createDirectories(Paths.get("outputs"));

        // store multi-level sort keys (first chosen is highest priority)
        List<SortKey> keys = new ArrayList<>();

        while (true) {
            // 2) Ask sort algorithm
            System.out.print("Choose sort algorithm [b=bubble, i=insertion]: ");
            String algo = sc.nextLine().trim().toLowerCase();
```

```
                algo = sc.nextLine().trim().toLowerCase();
            }

            // 3) Ask column (name or 0-based index)
            System.out.print("Sort by which column (name or 0-based index): ");
            String col = sc.nextLine().trim();
            if (col.isEmpty()) {
                System.out.println("Column cannot be empty. Try again.");
                continue;
            }

            // Add to chain (first chosen is primary)
            keys.add(new SortKey(algo, col));

            // Apply multi-level sorting:
            // To keep the FIRST key as highest priority, apply stable sorts in REVERSE order.
            for (int k = keys.size() - 1; k >= 0; k--) {
                SortKey sk = keys.get(k);
                int colIdx = db.resolveColumn(sk.col);
                if (sk.algo.equals("b")) db.bubbleSort(colIdx, true);
                else db.insertionSort(colIdx, true);
            }
            System.out.println("Sorted by keys (primary→secondary): " + describeKeys(keys));
```

```
            // 4) Options
            System.out.print("Options: [e=export+quit, ec=export+continue, c=continue, q=quit] : ");
            String opt = sc.nextLine().trim().toLowerCase();
            if (opt.equals("e") || opt.equals("ec")) {
                String outPath = makeExportPath(keys);
                try (BufferedWriter w = Files.newBufferedWriter(Paths.get(outPath))) {
                    db.exportRecursively(w);
                }
                System.out.println("Exported: " + outPath);
                if (opt.equals("e")) { System.out.println("Bye!"); break; }
                // else continue
            } else if (opt.equals("q")) {
                System.out.println("Bye!"); break;
            } else if (opt.equals("c") || opt.isEmpty()) {
                // loop to add another key
            } else {
                System.out.println("Unknown option; continuing.");
            }
        }
    }

    private static String describeKeys(List<SortKey> keys) {
        StringBuilder sb = new StringBuilder();
        for (int i=0;i<keys.size();i++) {
            if (i>0) sb.append(" > ");
            sb.append(keys.get(i).algo.equals("b") ? "bubble" : "insertion")
              .append("(").append(keys.get(i).col).append(")");
        }
        return sb.toString();
    }

    private static String makeExportPath(List<SortKey> keys) {
        String base = "outputs/sorted";
        for (SortKey k : keys) base += "_" + (k.algo.equals("b") ? "b" : "i") + "-" + sanitize(k.col);
        base += "_" + System.currentTimeMillis() + ".csv";
        return base;
    }
    private static String sanitize(String s) {
        return s.replaceAll("[^A-Za-z0-9_-]", "");
    }
}
```

5. Java Implementation – CsvUtils.java

```
import java.nio.file.*;
import java.util.*;

public class CsvUtils {
    public static class Table {
        public List<String> header;
        public List<List<String>> rows;
    }

    public static Table readCsv(String path) throws Exception {
        List<String> lines = Files.readAllLines(Paths.get(path));
        if (lines.isEmpty()) throw new IllegalArgumentException("Empty CSV: " + path);
        Table t = new Table();
        t.header = parseLine(lines.get(0));
        t.rows = new ArrayList<>();
        for (int i = 1; i < lines.size(); i++) {
            String ln = lines.get(i).trim();
            if (!ln.isEmpty()) t.rows.add(parseLine(lines.get(i)));
        }
        return t;
    }

    // CSV parse with quotes
    private static List<String> parseLine(String line) {
        List<String> out = new ArrayList<>();
        StringBuilder cur = new StringBuilder();
        boolean inQuotes = false;
        for (int i=0; i<line.length(); i++) {
            char c = line.charAt(i);
            if (inQuotes) {
                if (c=='"') {
                    if (i+1<line.length() && line.charAt(i+1)=='"') { cur.append('"'); i++; }
                    else inQuotes=false;
                } else cur.append(c);
            } else {
                if (c==',') { out.add(cur.toString()); cur.setLength(0); }
                else if (c=='"') inQuotes=true;
                else cur.append(c);
            }
        }
        out.add(cur.toString());
        return out;
    }
}
```

6. CPU & Disk Command Line

```
cd /home/balachv/IndividualHomework3/algorithms-hw3 || { echo "Project folder not found"; exit 1; }

# deps for iostat/pidstat (first run only; safe to re-run)
sudo apt-get update && sudo apt-get install -y sysstat

# where we keep logs
mkdir -p perf

# compile Java (safe to re-run)
mkdir -p java/out
javac -d java/out java/*.java

# ask once for the column (name or 0-based index)
read -rp "Column to sort by (name or 0-based index) [Age]: " COL
COL=${COL:-Age}
echo "Using column: $COL"

# ---- JULIA: bubble (NO EXPORT: 'q') ----
iostat -dx 1 10 > perf/iostat_julia_bubble.txt & IOP=$!
/usr/bin/time -v julia julia/main.jl 2> perf/julia_bubble_time.txt <<EOF
data/student-data.csv
b
$COL
q
EOF
wait $IOP

# ---- JULIA: insertion (NO EXPORT: 'q') ----
iostat -dx 1 10 > perf/iostat_julia_insertion.txt & IOP=$!
/usr/bin/time -v julia julia/main.jl 2> perf/julia_insertion_time.txt <<EOF
data/student-data.csv
i
$COL
q
EOF
wait $IOP

# ---- JAVA: bubble (NO EXPORT: 'q') ----
iostat -dx 1 10 > perf/iostat_java_bubble.txt & IOP=$!
/usr/bin/time -v java -cp java/out Main 2> perf/java_bubble_time.txt <<EOF
data/student-data.csv
b
$COL
q
EOF
wait $IOP

# ---- JAVA: insertion (NO EXPORT: 'q') ----
iostat -dx 1 10 > perf/iostat_java_insertion.txt & IOP=$!
/usr/bin/time -v java -cp java/out Main 2> perf/java_insertion_time.txt <<EOF
data/student-data.csv
i
$COL
q
EOF
wait $IOP

echo
echo "Measurements complete (no CSVs exported)."
echo "Logs saved to: perf/"
echo "  - perf/julia_bubble_time.txt"
echo "  - perf/julia_insertion_time.txt"
echo "  - perf/java_bubble_time.txt"
echo "  - perf/java_insertion_time.txt"
echo "  - perf/iostat_*.txt"
```